



UNIVERSIDADE CATÓLICA DE SANTOS

Ciência da Computação

Explora+

Entrega de Protótipo

Felipe Barbosa dos Santos

Felipe de Lima Monteiro

Lucas Carmona Neto

Lucas Cerqueira Galvão

João Gabriel Henriques Cardoso

Santos – SP, junho de 2026

Sumário

1	Introdução	2
1.1	Problema	2
1.2	Solução proposta	2
1.3	Stakeholder externo	2
2	Desenvolvimento	2
2.1	Metodologia e organização do time	2
2.2	Proposta inicial de arquitetura	3
2.3	Arquitetura implementada	3
2.3.1	Stack tecnológico	3
2.3.2	APIs externas utilizadas	3
2.3.3	Diagrama de Componentes	4
2.3.4	Diagrama de Implantação	4
2.4	Modelagem de dados	5
2.4.1	Diagrama Entidade-Relacionamento	5
2.4.2	Diagrama de Classes	7
2.5	Casos de Uso	8
2.6	Fluxos principais	9
2.6.1	Gerar Rota Turística	9
2.6.2	Enriquecimento de Detalhe de POI	10
2.6.3	Marcar como Visitado	10
2.6.4	Diagrama de Atividade	11
2.7	MVP Entregue	12
2.7.1	Funcionalidades implementadas	12
2.7.2	Funcionalidades fora do escopo atual	12
3	Conclusão	13
3.1	Justificativas técnicas das escolhas	13

1 Introdução

O projeto Explora+ é uma iniciativa de extensão desenvolvida por estudantes de Ciência da Computação da Universidade Católica de Santos com o objetivo de criar um aplicativo móvel para promoção do turismo local.

1.1 Problema

Cidades de médio porte como Santos – SP possuem rico acervo turístico e cultural, mas enfrentam baixa visibilidade de seus atrativos pela ausência de ferramentas digitais acessíveis e integradas. Visitantes e moradores frequentemente desconhecem monumentos históricos, parques e pontos de interesse próximos, e não têm como planejar percursos turísticos de forma automática.

1.2 Solução proposta

O Explora+ oferece um planejador de rotas turísticas que:

- Calcula automaticamente um percurso pedestre entre dois pontos;
- Identifica pontos de interesse (POIs) próximos ao trajeto via dados abertos do OpenStreetMap;
- Enriquece os detalhes de cada POI com imagens e resumos buscados no Wikidata e Wikipedia;
- Permite ao usuário marcar lugares como visitados, excluir paradas e manter uma biblioteca pessoal de lugares.

1.3 Stakeholder externo

O cliente externo do projeto é Felipe Pacheco Fernandes, CRO do Grupo Costa Norte de Comunicação, sediado na Baixada Santista/SP. O grupo atua como stakeholder, contribuindo com feedback sobre comunicação e facilitando contato com estabelecimentos comerciais e agências de turismo locais.

2 Desenvolvimento

2.1 Metodologia e organização do time

O time adotou metodologia ágil com sprints semanais, organização de tarefas via Trello e reuniões periódicas com o Product Owner externo. O repositório foi estruturado em três projetos independentes:

- `explora-plus-backend` – API Django;
- `explora-plus-frontend` – app Expo/React Native;
- `explora-plus-docs` – documentação, modelagem e paper.

2.2 Proposta inicial de arquitetura

No início do semestre, a arquitetura proposta previa integração com APIs comerciais como Google Maps (geolocalização e roteamento), GeoAPIFy (backup de geocodificação), Sympla e TicketMaster (eventos culturais), Uber e 99 (mobilidade urbana) e Quanto Tempo Falta (transporte público). O modelo de dados inicial era centrado em uma entidade `Route` de destino único (*single-destination*).

Durante o desenvolvimento, essa proposta foi substituída por uma arquitetura totalmente baseada em dados abertos, eliminando dependências de APIs pagas e ampliando a cobertura de POIs. As justificativas para as mudanças são detalhadas na Seção 2.3.

2.3 Arquitetura implementada

2.3.1 Stack tecnológico

Tabela 1: Stack tecnológico do Explora+

Camada	Tecnologia	Versão
Backend framework	Django + DRF	5.1.4 / 3.15.2
Autenticação	SimpleJWT	5.3.1
Banco de dados	PostgreSQL + PostGIS	16 / 3.4
Infraestrutura	Docker + Docker Compose	–
Frontend framework	Expo + React Native	52 / 0.76
Linguagem frontend	TypeScript	–
Mapa	Leaflet (via WebView/iframe)	–

2.3.2 APIs externas utilizadas

A decisão de usar exclusivamente APIs gratuitas e abertas foi motivada por três fatores: (1) sustentabilidade do projeto sem custos de operação; (2) disponibilidade imediata sem cadastro ou aprovação; (3) qualidade dos dados do OpenStreetMap para a região de Santos.

Tabela 2: APIs externas integradas

Serviço	Papel	Substitui
Nominatim	Geocodificação de endereços	Google Maps / GeoAPIFy
OSRM	Roteamento pedestre	Google Maps Directions
Overpass API	Busca de POIs via OSM	Google Places
Wikidata API	Imagem principal do lugar (P18)	–
Wikipedia REST API	Resumo descritivo e imagem fallback	–

O enriquecimento de detalhes de POI segue uma cadeia progressiva: Nominatim fornece endereço e extratags OSM (que podem conter `wikidata` e `wikipedia`); Wikidata fornece imagem via propriedade P18; se não houver título Wikipedia disponível, o sistema pesquisa o Wikipedia por nome do lugar (primeiro em português, depois em inglês) como fallback (OpenStreetMap Contributors, 2024a; Wikimedia Foundation, 2024a,b).

2.3.3 Diagrama de Componentes

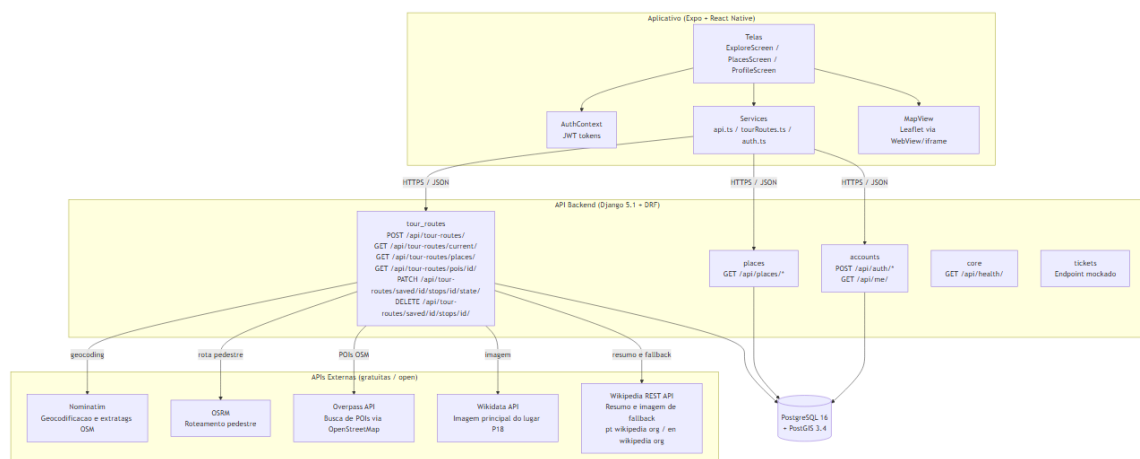


Figura 1: Diagrama de componentes do Explora+

2.3.4 Diagrama de Implantação

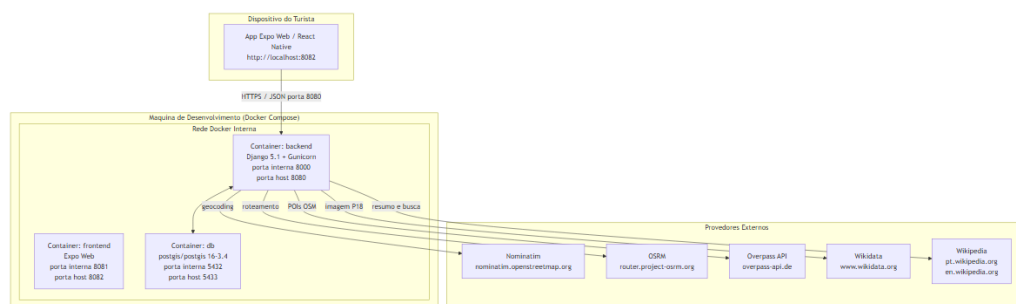


Figura 2: Diagrama de implantação – ambiente Docker de desenvolvimento

O ambiente de desenvolvimento usa Docker Compose com três containers: backend (Django + Gunicorn, porta 8080), frontend (Expo Web, porta 8082) e db (PostgreSQL + PostGIS, porta 5433) (Docker Inc., 2024).

2.4 Modelagem de dados

2.4.1 Diagrama Entidade-Relacionamento

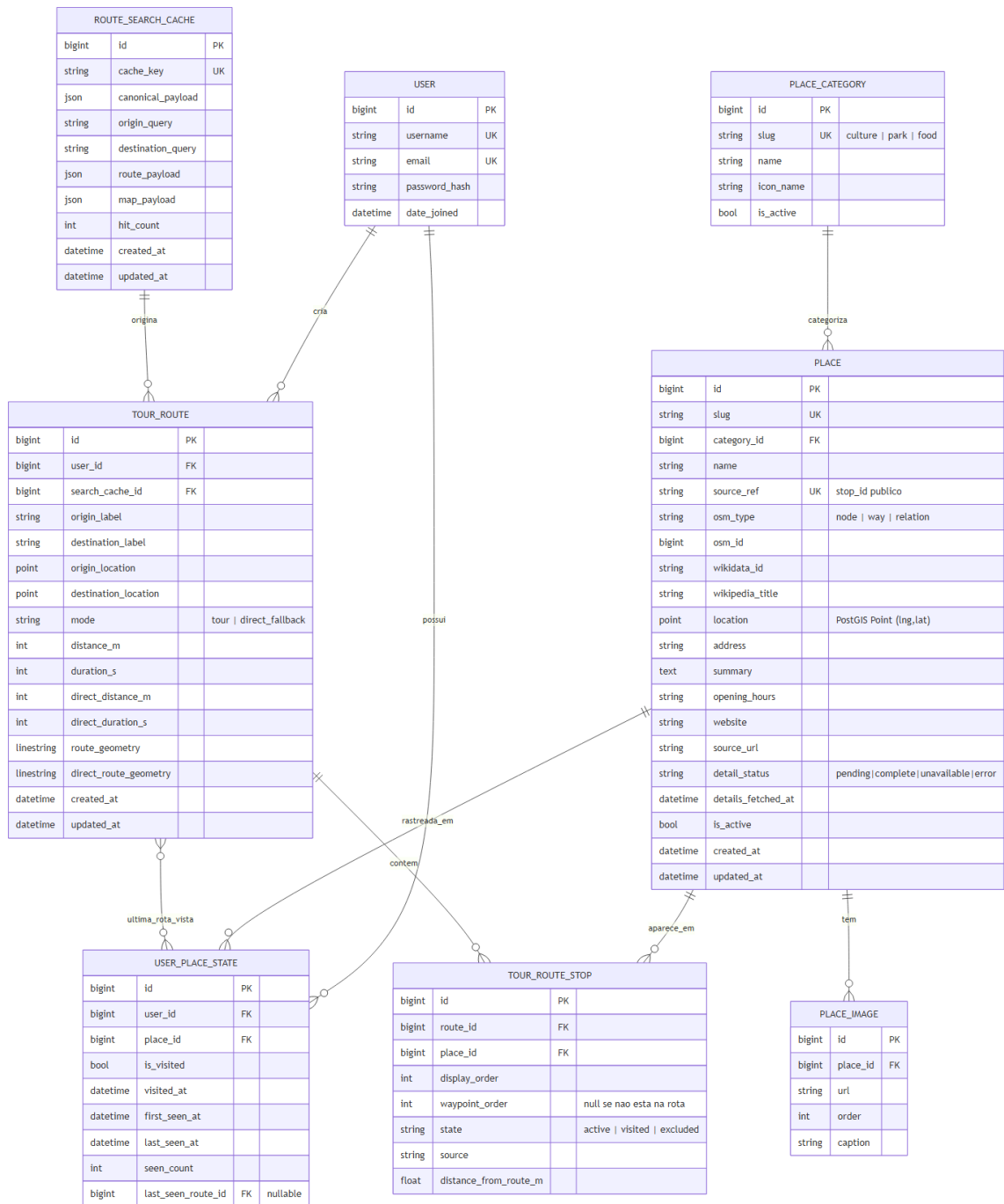


Figura 3: Diagrama ER do Explora+ – MVP entregue

O modelo de dados evoluiu significativamente em relação à proposta inicial. A entidade `Route` de destino único foi substituída por um conjunto de entidades que suportam rotas multi-POI e personalização por usuário:

- **Place:** fonte única de verdade de um POI. Campo `source_ref` é o identificador público (`stop_id` no contrato HTTP). Campo `detail_status` controla o ciclo de enriquecimento progressivo.
- **UserPlaceState:** estado global do usuário em relação a um lugar (visitado, contagens, última rota em que apareceu).
- **RouteSearchCache:** cache de rota calculada por chave canônica (origem + destino). Evita recalcular via Nominatim/OSRM/Overpass a cada requisição.
- **TourRoute:** snapshot relacional da rota personalizada do usuário com geometria `LineString` PostGIS.
- **TourRouteStop:** cada POI na rota com estado `active` / `visited` / `excluded`.

2.4.2 Diagrama de Classes

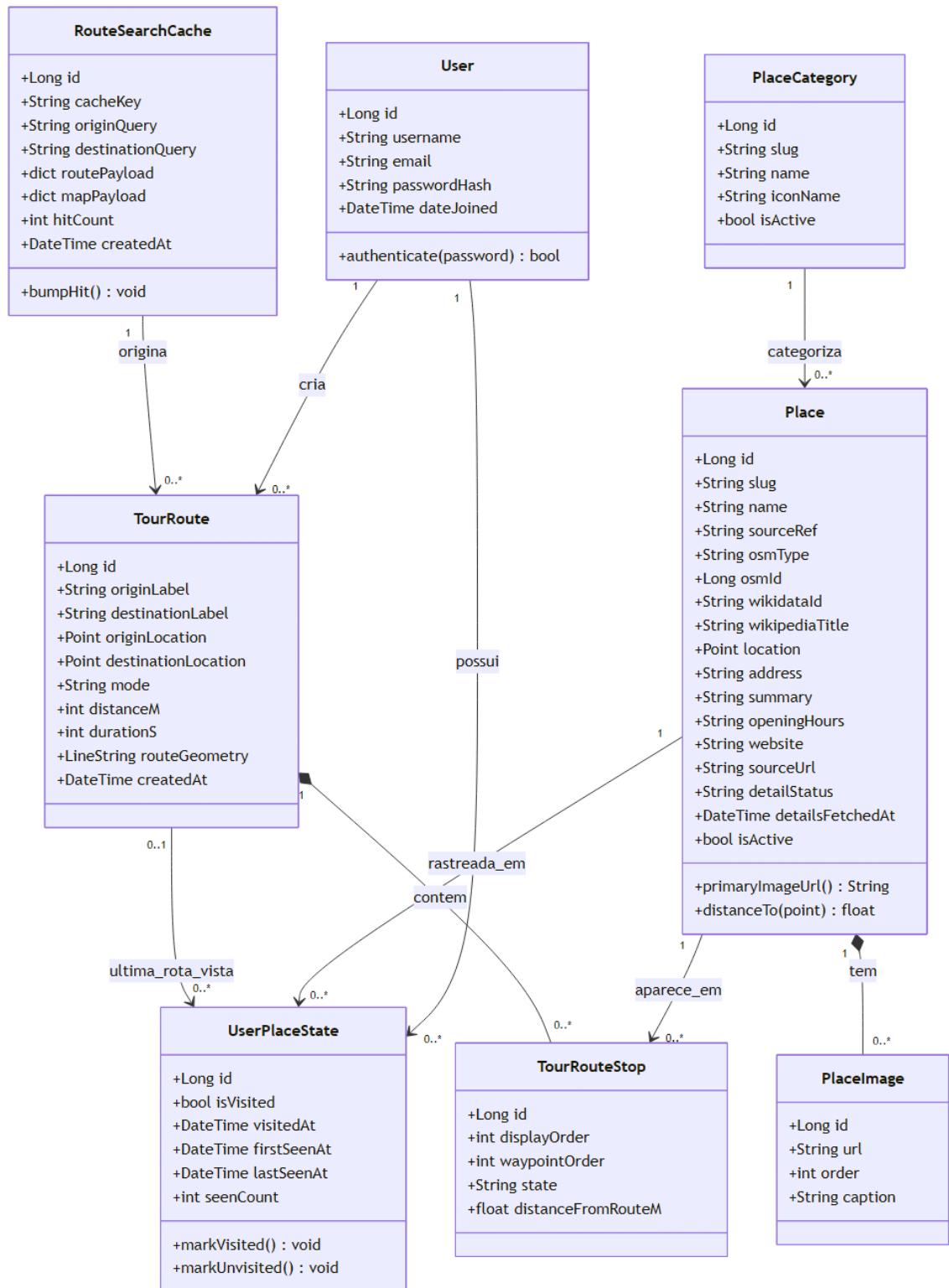


Figura 4: Diagrama de classes principal

2.5 Casos de Uso

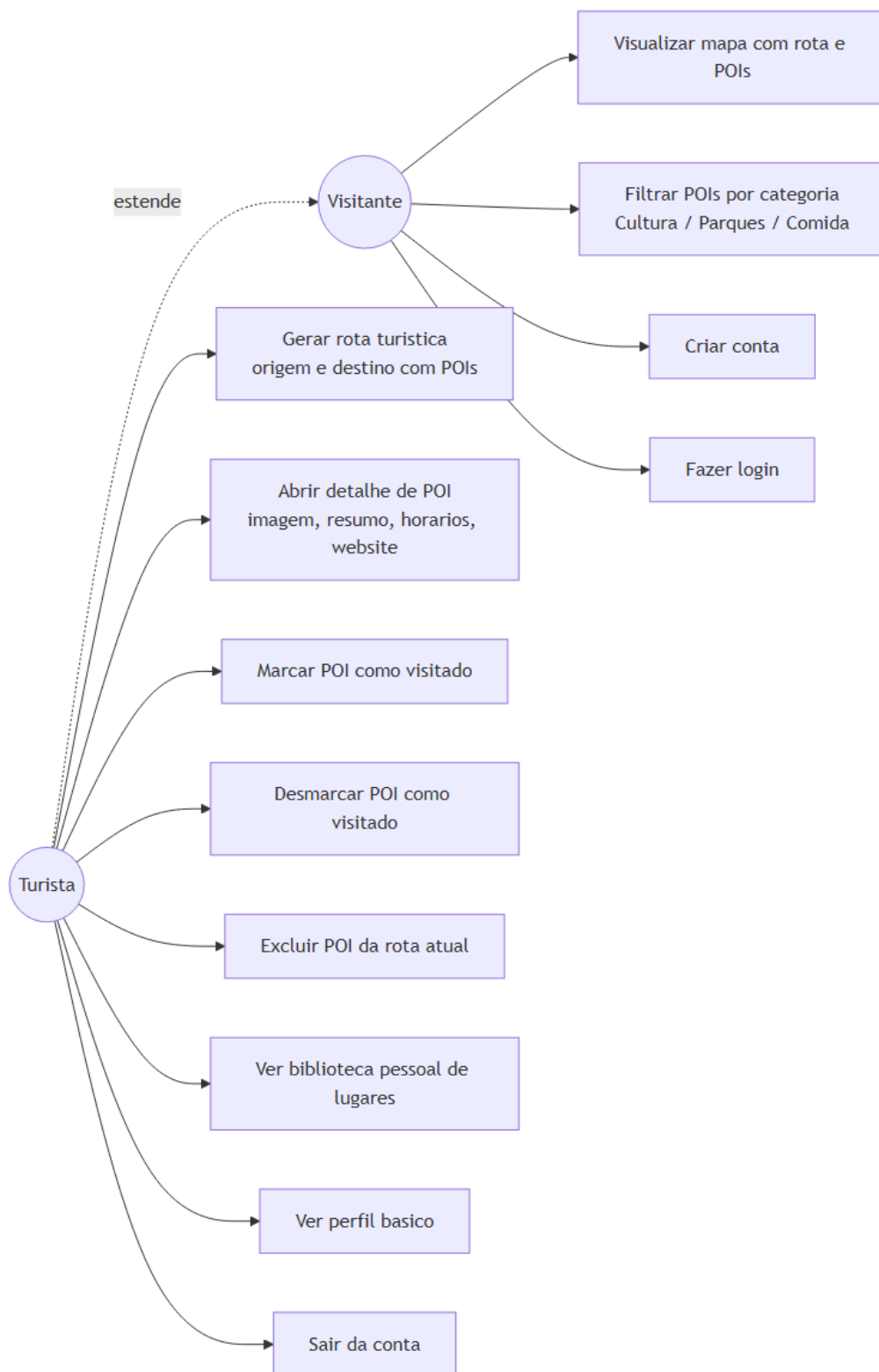


Figura 5: Diagrama de casos de uso – MVP

O MVP define dois atores: **Visitante** (usuário anônimo) e **Turista** (usuário autenticado, que estende Visitante). O ator Administrador da proposta inicial foi removido do escopo: lugares são populados via comando `seed_demo` e descobertos automaticamente pelo Overpass API.

2.6 Fluxos principais

2.6.1 Gerar Rota Turística

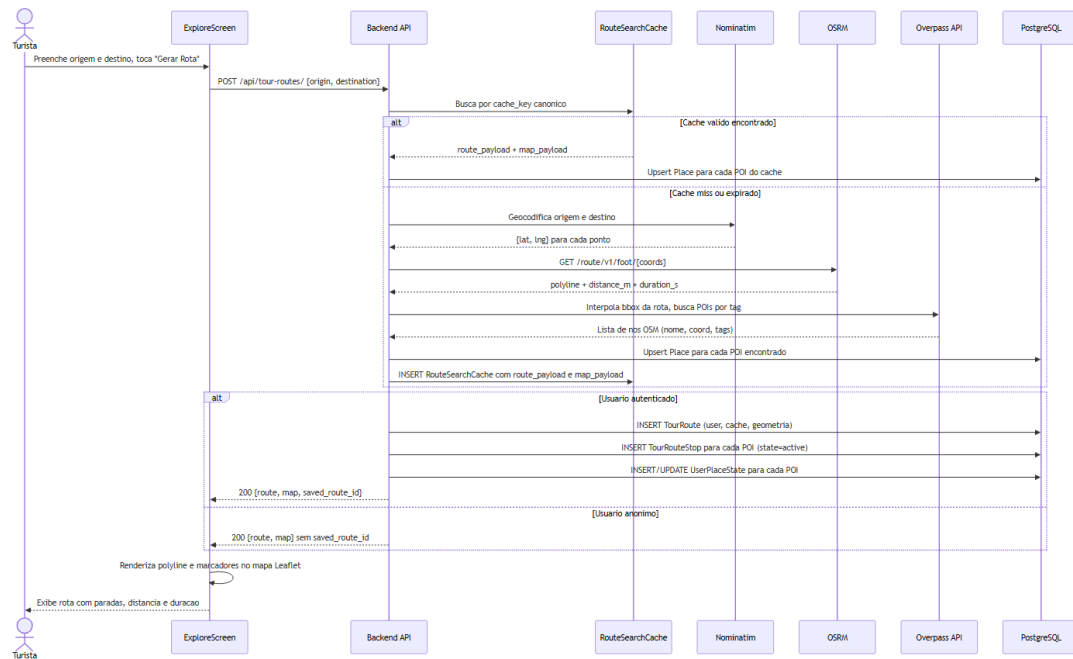


Figura 6: Diagrama de sequência – Gerar Rota

O fluxo de geração de rota é o núcleo do produto. Ao receber `POST /api/tour-routes/` com origem e destino, o backend: (1) verifica o cache por chave canônica; (2) em caso de miss, geocodifica via Nominatim, calcula rota pedestre via OSRM e busca POIs via Overpass; (3) realiza upsert dos POIs em Place; (4) se o usuário estiver autenticado, persiste TourRoute, TourRouteStop e UserPlaceState (Project-OSRM contributors, 2024; OpenStreetMap Contributors, 2024b).

2.6.2 Enriquecimento de Detalhe de POI

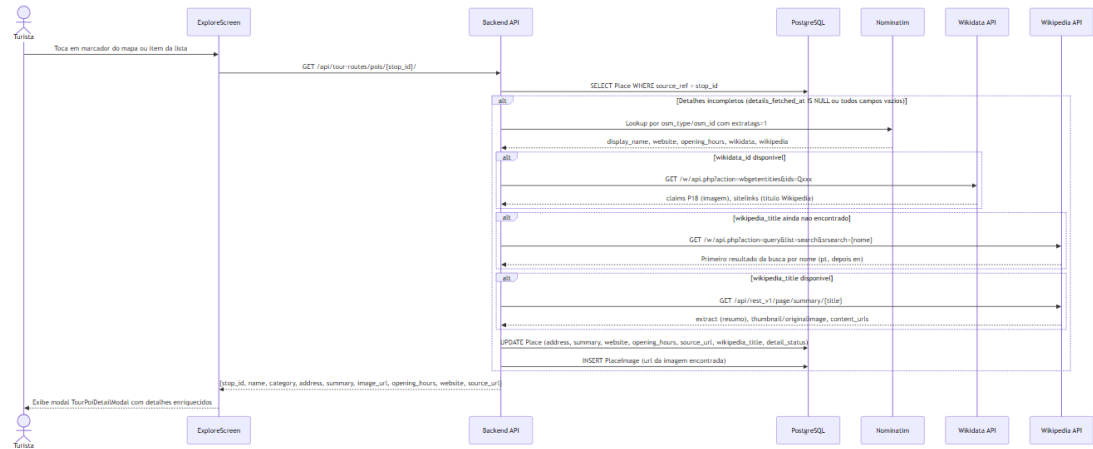


Figura 7: Diagrama de sequência – Enriquecimento de detalhe de POI

Ao abrir o detalhe de um POI (GET /api/tour-routes/pois/{stop_id}/), o backend verifica se os dados já foram enriquecidos. Se não, executa a cadeia: Nominatim (extratags OSM) → Wikidata (imagem P18) → Wikipedia por nome (fallback) → Wikipedia Summary API.

2.6.3 Marcar como Visitado

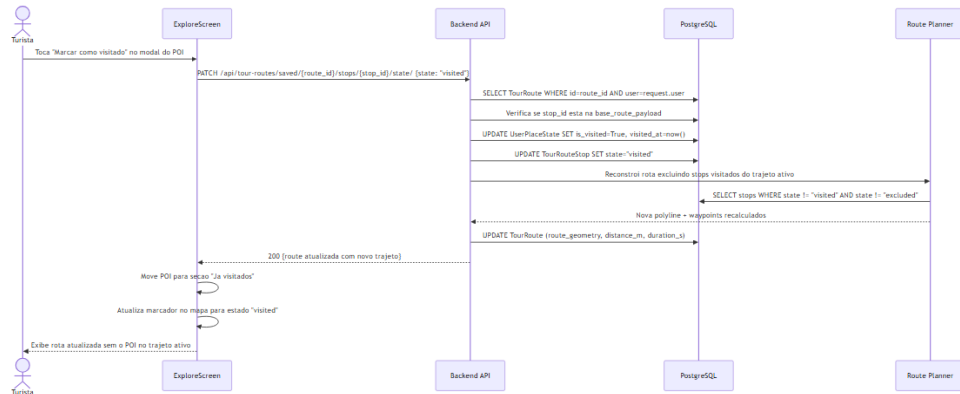


Figura 8: Diagrama de sequência – Marcar como Visitado

Ao marcar um POI como visitado (PATCH .../stops/{stop_id}/state/), o backend atualiza UserPlaceState e TourRouteStop, reconstrói a rota excluindo stops visitados do trajeto ativo, e devolve a rota atualizada.

2.6.4 Diagrama de Atividade

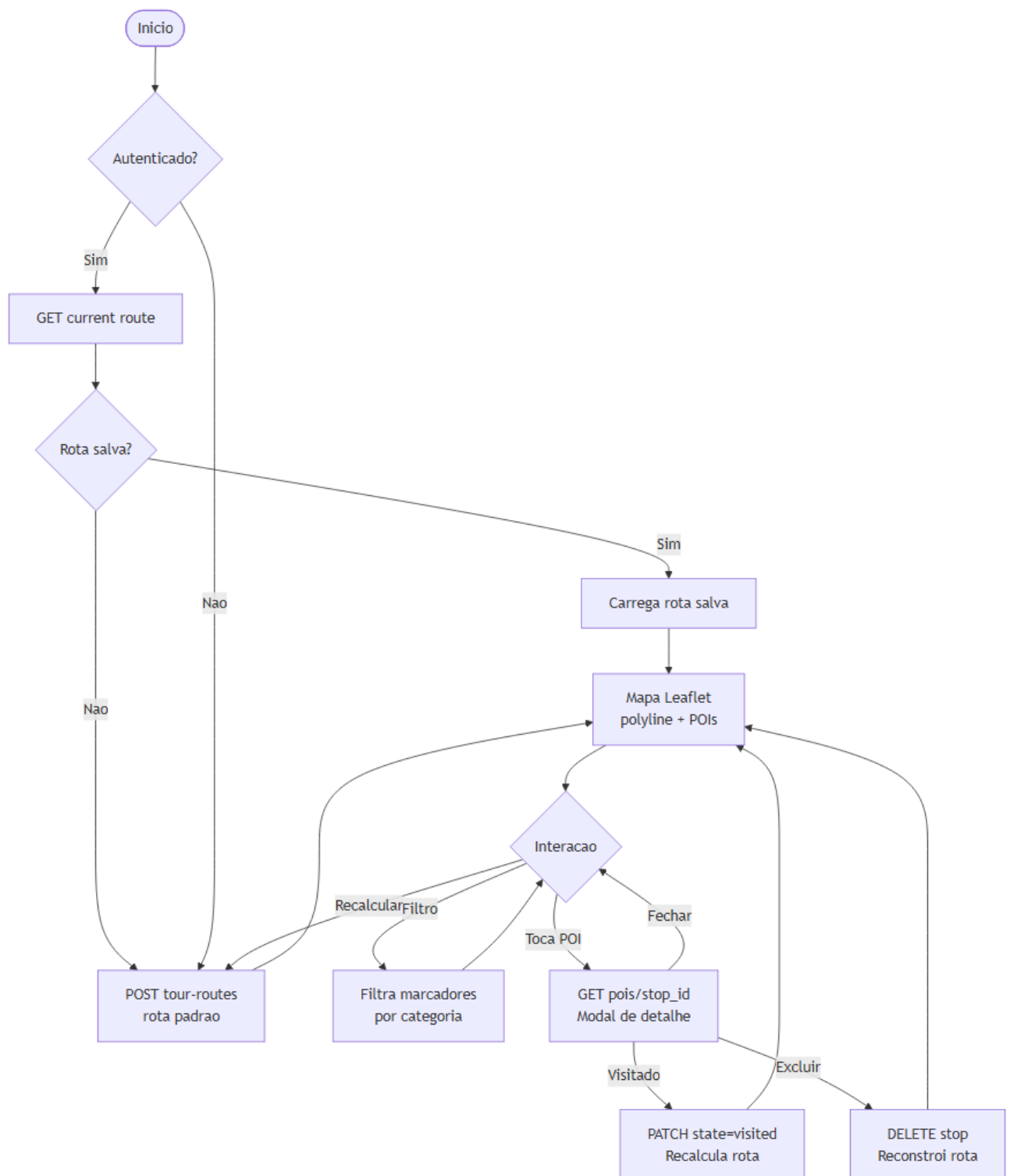


Figura 9: Diagrama de atividade – Fluxo da ExploreScreen

2.7 MVP Entregue

2.7.1 Funcionalidades implementadas

Tabela 3: Funcionalidades entregues no MVP

Funcionalidade	Descrição
Autenticação JWT	Registro, login, refresh de token e logout
Planejamento de rota	Origem + destino com POIs selecionados ao longo do percurso
Mapa interativo	Leaflet via WebView/iframe com polyline, marcadores e filtros
Filtros de categoria	Cultura, Parques, Comida
Detalhe de POI	Modal com imagem, resumo, endereço, horários, website
Enriquecimento automático	Imagem e resumo via Wikidata/Wikipedia
Marcar visitado	POI sai do trajeto ativo e vai para seção “Já visitados”
Excluir POI da rota	Rota recalculada sem o POI excluído
Biblioteca pessoal	Aba Lugares com histórico de todos os POIs vistos em rotas
Perfil básico	Dados do usuário e botão de logout
Cache de rotas	Rota reutilizada para mesma origem/destino sem recalcular

2.7.2 Funcionalidades fora do escopo atual

As seguintes funcionalidades foram identificadas na proposta inicial, mas não foram implementadas neste ciclo por decisão de escopo:

- Integração com eventos culturais (Sympla, TicketMaster);
- Integração com aplicativos de mobilidade (Uber, 99);
- Consulta de transporte público (Quanto Tempo Falta);
- Seleção de modal de transporte (apenas caminhada suportada);
- Compra real de ingressos (endpoint mockado);
- Edição de perfil de usuário;
- Busca por voz.

3 Conclusão

O Explora+ entregou um MVP funcional e coerente que cumpre o objetivo central do projeto: permitir que turistas e moradores descubram e percorram pontos de interesse em Santos com auxílio de um app móvel.

A principal mudança em relação à proposta inicial foi a substituição de APIs comerciais por alternativas abertas (Nominatim, OSRM, Overpass, Wikidata, Wikipedia), o que tornou o sistema mais sustentável e independente de cotas e custos. O modelo de dados evoluiu de uma rota *single-destination* para um planner multi-POI com cache, personalização por usuário e enriquecimento progressivo de conteúdo.

3.1 Justificativas técnicas das escolhas

React Native + Expo Gera aplicação nativa para Android, iOS e Web a partir de uma única base de código, reduzindo custo de manutenção (Meta Platforms, Inc., 2024; Expo Dev, 2024).

Django + DRF Maturidade do framework, mapeamento ORM direto das entidades de negócio e segurança integrada (Django Software Foundation, 2024; Encode OSS Ltd., 2024; Holovaty; Kaplan-Moss, 2009).

PostgreSQL + PostGIS Suporte nativo a operações geoespaciais (distância, proximidade, LineString) necessárias para roteamento e seleção de POIs (OSGeo, 2024).

OpenStreetMap + Nominatim + OSRM + Overpass Alternativa gratuita, global e com cobertura adequada para Santos. Elimina dependência de APIs pagas (OpenStreetMap Foundation, 2024; OpenStreetMap Contributors, 2024a; Project-OSRM contributors, 2024; OpenStreetMap Contributors, 2024b).

Wikidata + Wikipedia Fonte rica de imagens e descrições culturais para pontos de interesse, sem necessidade de curadoria manual (Wikimedia Foundation, 2024a,b).

Docker + Docker Compose Ambiente de desenvolvimento idêntico para todos os integrantes, com hot reload e isolamento de dependências (Docker Inc., 2024; Humble; Farley, 2010).

Referências

DJANGO SOFTWARE FOUNDATION. **Django documentation**. [S. l.: s. n.], 2024. <https://docs.djangoproject.com>. Acesso em: jun. 2026.

DOCKER INC. **Docker documentation**. [S. l.: s. n.], 2024. <https://docs.docker.com>. Acesso em: jun. 2026.

ENCODE OSS LTD. **Django REST Framework documentation**. [S. l.: s. n.], 2024.
<https://www.django-rest-framework.org>. Acesso em: jun. 2026.

EXPO DEV. **Expo documentation**. [S. l.: s. n.], 2024. <https://docs.expo.dev>.
Acesso em: jun. 2026.

HOLOVATY, Adrian; KAPLAN-MOSS, Jacob. **The definitive guide to Django: web development done right**. 2. ed. New York: Apress, 2009.

HUMBLE, Jez; FARLEY, David. **Continuous delivery: reliable software releases through build, test, and deployment automation**. Boston: Addison-Wesley, 2010.

META PLATFORMS, INC. **React Native documentation**. [S. l.: s. n.], 2024.
<https://reactnative.dev/docs/getting-started>. Acesso em: jun. 2026.

OPENSTREETMAP CONTRIBUTORS. **Nominatim — OpenStreetMap geocoding service**. [S. l.: s. n.], 2024. <https://nominatim.openstreetmap.org>. Acesso em: jun. 2026.

OPENSTREETMAP CONTRIBUTORS. **Overpass API**. [S. l.: s. n.], 2024.
<https://overpass-api.de>. Acesso em: jun. 2026.

OPENSTREETMAP FOUNDATION. **OpenStreetMap**. [S. l.: s. n.], 2024.
<https://www.openstreetmap.org>. Acesso em: jun. 2026.

OSGEO. **PostGIS documentation**. [S. l.: s. n.], 2024.
<https://postgis.net/documentation>. Acesso em: jun. 2026.

PROJECT-OSRM CONTRIBUTORS. **OSRM — Open Source Routing Machine**. [S. l.: s. n.], 2024. <https://project-osrm.org>. Acesso em: jun. 2026.

WIKIMEDIA FOUNDATION. **Wikidata**. [S. l.: s. n.], 2024.
<https://www.wikidata.org>. Acesso em: jun. 2026.

WIKIMEDIA FOUNDATION. **Wikipedia REST API**. [S. l.: s. n.], 2024.
https://en.wikipedia.org/api/rest_v1/. Acesso em: jun. 2026.